

Chess Attribute Classifier System Specification

1. Overview

This document specifies a modular system consisting of four independent classifiers for extracting chess piece attributes from camera photos of screen-rendered chessboards. The system does NOT rely on FEN or internal chess representations.

Target classifiers:

- Color classifier: 0 = none, 1 = side-A piece, 2 = side-B piece.
- Type classifier: pawn, knight, bishop, rook, queen, king.
- X classifier: board file index (1..8).
- Y classifier: board rank index (1..8).

2. System Architecture

The system is divided into preprocessing and four independent classifiers.

Step 1: Board corner detection (keypoint model).

Step 2: Homography warp to canonical square board image.

Step 3: Split warped board into 8x8 square crops.

Step 4: Train and run independent classifiers on square crops.

Note: X and Y are deterministically known after grid splitting, but separate classifiers may still be trained if required by system design constraints.

3. Datasets and References

Board Corner Detection Dataset:

- surawut/chessboard-dataset-yolo (HuggingFace):
<https://huggingface.co/datasets/surawut/chessboard-dataset-yolo>
- Roboflow Chessboard Corners dataset:
<https://universe.roboflow.com/chessboarddetect/chessboard-corners-wwivs>

Piece and Board Rendering Datasets:

- MohammedHemed/Chessboard-digital-images_with_fen (HuggingFace):
https://huggingface.co/datasets/MohammedHemed/Chessboard-digital-images_with_fen
- Synthetic Chess Board Images (Kaggle):
<https://www.kaggle.com/datasets/thefamousrat/synthetic-chess-board-images>
- ChessRender360 (Kaggle):
<https://www.kaggle.com/datasets/mmkoya/chessrender360>

4. Model Specifications

Color Model:

- Backbone: ResNet-18 or EfficientNet-B0
- Output: 3-class softmax
- Loss: Cross-entropy

Type Model:

- Backbone: ResNet-18
- Output: 6-class softmax (or 7-class including 'none')
- Loss: Cross-entropy

X Model:

- Input: Square crop (or local 3x3 patch)
- Output: 8-class softmax
- Loss: Cross-entropy

Y Model:

- Same structure as X model.

5. Data Augmentation Strategy

To bridge the domain gap between clean digital datasets and camera photos, apply the following augmentations during training:

- Perspective distortion
- Motion blur
- JPEG compression artifacts
- Brightness and contrast shifts
- Glare and bloom effects
- Moiré / scanline simulation
- Partial occlusions (UI overlays, cursor highlights)

6. Suggested Repository Structure

```
chess_attr/  
  data/  
  models/  
  scripts/  
    01_download_datasets.py  
    02_train_corners.py  
    03_warp_and_crop.py  
    04_build_labels.py  
    05_train_color.py  
    06_train_type.py  
    07_train_x.py  
    08_train_y.py  
    09_infer.py
```